Aufgabe A1.1: Grundlagen von ROS2

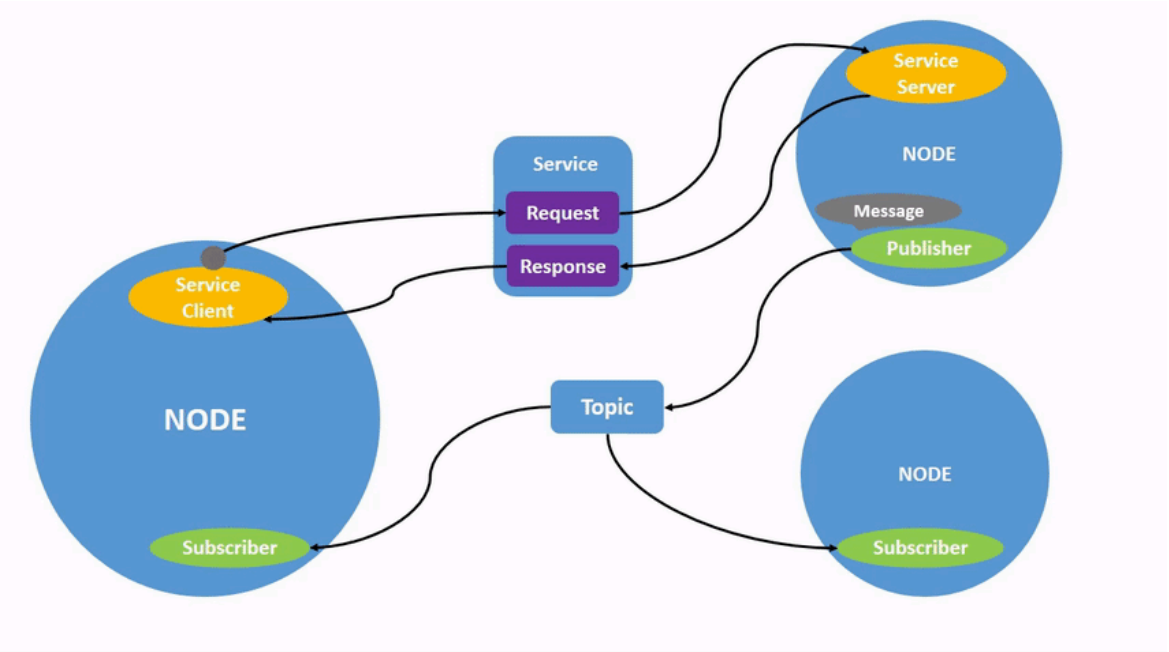
1. Wie wird das Programm aufgebaut?

Workspace:

Ein Workspace ist ein **Verzeichnis**, das den Quellcode, Build-Dateien und Installationen eines ROS2-Projektes enthält. Typische Ordner sind `src`, `build`, `install`, `log`. Im `src`-Verzeichnis entstehen alle benötigten Packages.

Package:

Ein Package ist die kleinste organisatorische **Einheit** in ROS2 und enthält Nodes, Libraries, Konfigurationsdateien, und URDF-Modelle. Ein Projekt besteht meistens aus mehreren Packages.



Topic:

Topics steuern die Verbindung zwischen den Nodes. Sie bestimmen, welche Node mit welcher anderen Node kommuniziert und ist der Kommunikationskanal für den Nachrichtenaustausch zwischen den Nodes. Es können eine oder mehrere Nodes als subscriber oder publisher an den Topic verbunden sein. Wenn ein Publisher (Node) eine Message an den Topic sendet, empfangen alle Subscriber diese Message.

Message:

sind die Daten, die zwischen den Nodes über Topics ausgetauscht werden. Eine Node veröffentlicht zu einem Topic eine Nachricht, und alle Nodes, die diesen Topic abonniert haben, erhalten die Nachricht.

Message typen
ROS-Pakete bieten neben ihrer Funktionalität auch verschiedene Nachrichtentypen an. Einige der am häufigsten verwendeten Pakete sind:

- **std_msgs** – Nachrichten mit grundlegenden Datentypen wie Integer oder float
- **geometry_msgs** – Zum Verarbeiten von Informationen über Position, Orientierung, Geschwindigkeit oder Beschleunigung von Objekten sowie zur Definition von Punkten oder Polygonen
- **nav_msgs** – Zum Verarbeiten von Karten oder Lokalisierungsdaten von dem Roboter
- **sensor_msgs** – Zum Verarbeiten von Sensordaten

Jedes dieser Pakete enthält viele Nachrichtentypen, die in unterschiedlichen Anwendungen genutzt werden.
Beim Erstellen von Nodes für mobile Roboter wird man wahrscheinlich einigen dieser Nachrichten begegnen, wie zum Beispiel:

- **geometry_msgs/Twist** – Eine Nachricht zur Definition von linearer und angularer Geschwindigkeit eines Objekts. Kann auch verwendet werden, um gewünschte Geschwindigkeiten vorzugeben.

source:
https://husarion.com/tutorials/ros2-tutorials/2-creating-nodes-messages/#message-types

Node:

Eine Node ist ein einzelnes Programm innerhalb eines ROS2-Systems. Eine Node ist dafür da, Aufgaben zu erledigen, wie z.B. Daten vom Laserscanner verarbeiten oder um zu erkennen, wo ein Objekt steht. Eine Node kann Messages publishen und subscriben (via topics oder services).

Wie läuft der Build eines ROS2-Workspace ab ?

1. ROS muss zuvor Installiert werden mit "**source /opt/ros/humble/setup.bash**"
2. Ordner für den Workspace mit z.b "**mkdir ros2_ws/src**" erstellen
3. In das Verzeichnis gehen und die ROS 2 Pakete in den src Ordner einfügen (entweder rein kopieren oder mit git clone)
4. Abhängigkeiten auflösen mit "**rosdep install -i --from-path src --rosdistro humble -y**" im **ros2_ws** Ordner
5. Build mit "**colcon build**" im **ros2_ws** Ordner (falls noch nicht installier. Mit **sudo apt install python3-colcon-common-extensions**)

   Wir haben **colcon build --packages-select rosy_description** benutzt als wir den Workspace gebaut haben

6. Mit "**source /opt/ros/humble/setup.bash**" das Environment einrichten
7. Overlay im ros2_ws mit "**source install/local_setup.bash**" erstellen (damit werden alle Pakete im Workspace zur ROS-Umgebung hinzugefügt)
8. Optional kann man auch in die ***.bashrc** File die Befehle einfügen damit sie automatisch beim öffnen eines Terminals ausgeführt werden.

Was sind Launch Files?

Launch Files sind Dateien, welche beschreibt

- welche Nodes gestartet werden sollen
- welche Parameter sie haben (z.b. 'max_velocity': 1.0)
- in welchen Namespace (Namespace gruppiert Topics, Services und Nodes z.b. Robot1)
- ob es Remapping gibt von Topics (z.b. remappings=['/laser', '/laser_1'])
- ob es Delays, oder andere Bedingungen gibt (z.b. verspäteter start period=5.0, action=[...])

Aufgabe A1.2: URDF

Die Visualisierung

1. Klonen des Repositories (dieses Repository ist das Workspace):

   bash
   ```
   git clone https://github.com/MusLead/gr4_ue1_ws.git
   cd rosy_ws
   ```

2. Workspace bauen:

   bash
   ```
   colcon build
   ```
   Oder
   ```
   colcon build --packages-select rosy_description
   ```

3. Umgebung einrichten:

   bash
   ```
   source install/setup.bash
   ```

4. Visualisierung starten:

   bash
   ```
   ros2 launch rosy_description display.launch.py
   ```

Was das Projekt macht

- Lädt das URDF-Modell des Roboters in **RViz2**.
- Verwendet **joint_state_publisher_gui**, um die Drehung der Radgelenke des Roboters zu simulieren.
- Ideal, um das Robotermodell und seine Bewegungen vor dem realen Einsatz zu überprüfen.

Referenz: Erstellen von einem ROS 2-Paket

bash
```
ros2 pkg create rosy_description
```

Ersetze `rosy_description` durch den gewünschten Paketnamen.

source:
Create and Set Up a ROS2 Workspace - ROS2 Tutorial 3
ROS2 - How to Publish TFs using URDF and Robot State Publisher
Create a URDF with ROS2 [1H Crash Course]
https://articulatedrobotics.xyz/tutorials/mobile-robot/concept-design/concept-urdf/

In der Präsentation werden das Modell des Roboters und der Aufbau des Packages erklärt.

Aufgabe A1.2: URDF

1. Welche Messages werden auf welchen Topics zur Ansteuerung generiert und wie sind sie aufgebaut?

Die Teleop-Node (Tastatursteuerung) sendet Steuerbefehle an den ROSBot über das Topic:

- **Topic:** /cmd_vel
- **Message-Typ:** geometry_msgs/msg/Twist

Aufbau der Nachricht:

```
# This expresses velocity in free space broken into its linear and angular parts.

Vector3 linear:
   x: <vorwärts/rückwärts-Geschwindigkeit>
   y: 0.0
   z: 0.0
Vector3 angular:
   x: 0.0
   y: 0.0
   z: <Drehgeschwindigkeit um Z-Achse>
```

Erklärung:

- Vorwärtsbewegung: linear.x > 0, angular.z = 0
- Drehung nach rechts: linear.x = 0, angular.z < 0

Beispiel aus den Ausgaben:

```
linear:
        x: 0
        y: 0
        z: 0
angular:
        x: 0
        y: 0
        z: 0
```

Aus diesem Beispiel können wir wissen, dass der Roboter sich nicht bewegt.

Diese Nachrichten werden kontinuierlich gesendet, solange eine Message an den Topic /cmd_vel geschickt wird. Die Datei wird dann weitergeleitet an Node /rosbot_base_controller, weil der Topic subscribe den Topic cmd_vel. In dem Node /rosbot_base_controller wird die Messages bearbeitet wie Steuerung von Roboter.

/teleop_twist_keyboard
/cmd_vel
/rosbot_base_controller

2. Welche weiteren Topics gibt es und welche Daten werden dort veröffentlicht?

Einige der wichtigsten Topics und deren Inhalte sind:

Topic	Was analysieren?	Worauf achten?
/odom	pose und twist	Position, Orientierung und Bewegungsgeschwindigkeit
/scan	ranges Array	Abstandswerte in verschiedenen Richtungen
/imu	orientation, angular_velocity, linear_acceleration	Sensordaten zur Bewegung
/diagnostic	level and values	Status vom Roboter

Beispiel von anderen Topic:

```
ros2 topic echo /camera/color/image_raw :
header:
        stamp:
                sec: 174…
                nanosec: 686…
        frame_id: camera_color_optical_frame
height: 480
width: 640
encoding: rgb8
is_bigendian: 0
step: 1920
data:
…
```

Bedeutung der Felder

- header
- stamp – Zeitstempel, wann das Bild aufgenommen wurde. Wichtig für Synchronisierung mit anderen Sensoren.
- frame_id – Referenzrahmen des Bildes. Zeigt, von welcher Kamera (bzw. welchem Koordinatensystem) das Bild stammt – hier: camera_color_optical_frame.
- height und width: Die Auflösung des Bildes in Pixeln – hier 480x640.
- encoding
  Gibt das Farbformat an – rgb8 bedeutet: 8 Bit pro Farbkanal (Rot, Grün, Blau), also ein normales Farbbild.
- is_bigendian: Gibt an, wie die Bytes im Speicher angeordnet sind. Meist 0 = little-endian (Standard bei PCs).
- step
  Anzahl der Bytes pro Zeile – hier 1920. Bei rgb8 = 3 Bytes pro Pixel × 640 Pixel = 1920.
- data

Das eigentliche Bild – eine lange Liste von Bytewerten (0–255), die die Farben jedes Pixels kodieren. Diese Daten sind binär, daher sieht man beim Echo meist nur Punkte oder gekürzte Darstellung.

Weitere Befehle
ros2 topic list -t (mit topic type) — topics aller sensoren und dessen Info
ros2 topic echo /cmd_vel — Ausgabe von dem Topic
ros2 topic info /name_topic — Infos zum Topic /name_topic
ros2 interface show /name_of_type — Infos zur Daten Typen
ros2 node info /name_node — Information zu dem /name_node

In der Präsentation wird gezeigt, dass einige Data-Typen von Topics und Beispiel von der Ausgabe.

Aufgabe B1.2: Simulation des Roboters

Simulation des ROSBot – Übersicht

1. Bestandteile der Simulation

- **Weltmodell (Environment):**

  Die Umgebung ist im Verzeichnis roblab gespeichert und wird in einer .sdf-Datei beschrieben. Sie wurde mithilfe von Meshes und .stl-Dateien modelliert, um z. B. Tische, Stühle und andere Laborobjekte realistisch darzustellen.

- **Roboter-Modell:**

  Das Modell des ROSBots befindet sich in den Verzeichnissen rosbot_description und rosbot_component_description. Die URDF- und Xacro-Dateien definieren Sensoren, Räder, Gelenke und das visuelle Erscheinungsbild des Roboters.

- **Entity Tree in Gazebo:**

  Listet alle Objekte in hierarchischer Form – zur Übersicht, nicht zur tiefergehenden Analyse gedacht.

2. Steuerung und Visualisierung

- **Teleoperation:**

  Mit dem Befehl

  ros2 run teleop_twist_keyboard teleop_twist_keyboard

  kann der Roboter manuell gesteuert werden.

- **Mapping in RViz:**

  Die Kartierung wird mit folgendem Befehl gestartet:

  ros2 launch rosbot_navigation rosbot_mapping.launch.py

  Die Karte wird live in RViz angezeigt. Bewegungen sind nur in bereits erkundeten (freien) Bereichen möglich.

3. Übertragbarkeit auf den echten Roboter

In der Theorie sollte sowohl in Simulation als auch auf dem echten Roboter dieselben ROS-Topics (wie cmd_vel, scan, map) verwendet werden, ist ein Wechsel einfach möglich. Der Roboter sollte sich in Simulation und in der realen Welt bewegen.

Beispiel:

Ein cmd_vel-Topic wird in der Simulation vom Teleop-Node gesendet und vom Gazebo-Roboter empfangen. Derselbe Topic kann bei einem echten ROSBot vom Motorcontroller abonniert werden – ohne die Steuerlogik zu ändern.